NEUST Bike Go: A Location-Based Bike Transportation and Booking Application for NEUST Students

Group Members:
Lopez, Francheska Claire L
Maglaque Jay-R D.
Manabat, Margie C.
Pascual Seigi R.

Problem

The group identified the following problems experienced by the NEUST community, including students, faculty members, and university employees, when using bike or pedicab transportation within the campus:

1. **Competition for Available Bikes During Morning Peak Hours** – During morning rush hours, many members of the NEUST community compete for available bikes because they are in a hurry to reach their classes, offices, or workplaces on time. Even when there is a queue, some passengers still try to get a ride ahead of others, making the process less organized.
2. **Difficulty Finding Available Rides During Dismissal Hours** – Passengers from distant buildings, particularly CICT, often depend on arriving bikes when going home. However, these bikes are sometimes occupied by other passengers before those waiting can get a ride. Because of this, some passengers resort to calling riders to arrange a pickup.
3. **Difficulty Identifying Previous Rides and Recovering Lost Belongings** – Passengers sometimes leave their belongings inside bikes or pedicabs, especially when they are in a hurry. However, they may forget the bike number or the rider they previously traveled with, making it difficult to identify the vehicle and recover their belongings.

Proposed Solution

To address the identified problems, the group proposes NEUST Bike Go, a mobile-based bike transportation application designed for the NEUST community, including students, faculty members, and university employees.

Inspired by ride-hailing applications like Grab, the system allows passengers to book a bike or pedicab using their mobile devices instead of relying only on waiting for available rides. Users can pin their preferred pickup location anywhere within the campus and choose where the bike will come from, either the NEUST Main Gate or the Architecture (Arki) area.

The application also includes a live tracking feature that allows passengers to monitor the bike's current location and see how far it is from their pickup point. A messaging feature is also provided so passengers can communicate directly with their assigned riders regarding pickup arrangements.

Additionally, the system keeps records of previous rides, including the rider's details and bike number. This makes it easier for passengers to identify the vehicle in case they accidentally leave their belongings behind.

Through these features, NEUST Bike Go aims to provide a more convenient and organized way of requesting bike transportation, especially during morning rush hours and dismissal periods.

Target Users

- **Students** – NEUST students who need convenient and accessible bike or pedicab transportation when traveling between campus buildings or going home.
- **Faculty Members** – Teaching personnel who need transportation within the campus, particularly when traveling between distant buildings.
- **University Employees** – Administrative and non-teaching personnel who need convenient transportation when moving around the university premises.
- **Bike Riders** – Individuals who provide bike or pedicab transportation services within the NEUST campus and accept ride requests through the application.

Application Features

NEUST Bike Go provides different features for passengers, pedicab drivers, and administrators to make campus transportation more convenient, accessible, and organized. Each feature is designed to support the booking process, communication, ride monitoring, and management of campus transportation services.

A. Student/Passenger Features

1. **Ride Booking** – Allows passengers to request a pedicab through the application without having to wait physically in line for an available ride.
2. **Pickup Location Pinning** – Allows passengers to pin their preferred pickup location anywhere within the campus using the map.
3. **Drop-off Location Selection** – Allows passengers to select their desired destination or drop-off location within the campus.
4. **Bike Origin Selection** – Allows passengers to choose where their pedicab will come from, either the NEUST Main Gate or the Architecture (Arki) area.
5. **Available Bike Viewing** – Allows passengers to view available bikes and their queue status at the Main Gate and Arki waiting areas.
6. **Live Location Tracking** – Allows passengers to monitor the current location of their booked pedicab through the map while waiting for their ride.

7. **In-App Messaging and Calling Mobile Number** – Allows passengers to communicate directly with their assigned drivers through messages or phone calls for easier coordination.
8. **Notifications** – Sends notifications to passengers when their booked pedicab is ready or has arrived at their pickup location.
9. **Ride History** – Allows passengers to view their previous trips, including the bike number, driver's name, fare paid, pickup location, and drop-off location. This also helps passengers identify the vehicle in case they accidentally leave their belongings behind.

B. Pedicab Driver Features

1. **Booking Requests** – Allows drivers to receive and accept ride requests from passengers through the application.
2. **Passenger and Trip Details** – Displays important booking information, including the passenger's name, pickup location, and drop-off location.
3. **In-App Messaging and Calling** – Allows drivers to communicate directly with passengers through messages or phone calls regarding pickup arrangements and other ride-related concerns.
4. **Earnings Monitoring** – Allows drivers to view and monitor their earnings from completed trips.
5. **Notifications** – Provides notifications about incoming booking requests and other important ride-related updates.
6. **GPS and Map Navigation** – Allows drivers to view their current location and navigate toward the passenger's pickup point and destination through the map.
7. **Online and Offline Status** – Allows drivers to set their availability to online when they are ready to accept bookings and offline when they are unavailable. This helps passengers identify which drivers are available for ride requests.

C. Admin Features

1. **Dashboard** – Displays an overview of the campus transportation system, including the total number of bikes, available bikes, active bikes, drivers, students, and completed or ongoing trips.
2. **Campus GPS and Location Management** – Allows administrators to manage campus locations, including designated pickup points, buildings, and geofence zones.
3. **Bicycle Fleet Management** – Allows administrators to view and manage registered bikes, assigned drivers, bike numbers, passenger capacity, and availability status.

4. **Trip Monitoring** – Allows administrators to monitor ongoing and completed trips, including trip details, assigned drivers, and bike numbers.
5. **Passenger Demand Monitoring** – Displays the number of waiting passengers at different campus locations to help administrators identify areas with high transportation demand, especially during peak hours.
6. **Demand Heatmap and Analytics** – Provides a visual representation of passenger demand and busy campus locations to help administrators monitor transportation needs and identify areas that require more available bikes.
7. **Maintenance Management** – Allows administrators to view reported bike problems, monitor maintenance requests, and update their repair status.
8. **Fare Management** – Allows administrators to set and update the standard fare for campus transportation.
9. **Broadcast Notices** – Allows administrators to send announcements and important updates to passengers and drivers.

Technical Implementation

NEUST Bike Go is developed using Ionic/Angular and Capacitor to provide a mobile-based transportation system for the NEUST community. It integrates mobile device capabilities, data management, REST API operations, and custom campus routing to support communication and ride coordination between passengers and pedicab drivers.

A. Capacitor Capabilities (Native Mobile Features)

The application uses the following Capacitor plugins and mobile capabilities:

1. **Live Device GPS (@capacitor/geolocation)** – Used to access the device's current GPS location and track the locations of passengers and drivers. It is implemented through campus-geolocation.service.ts, with browser navigator.geolocation as a fallback.
2. **Local Notifications (@capacitor/local-notifications)** – Used to send alerts about important application events, such as new booking requests, driver arrivals, and chat messages. It is implemented through notification.service.ts.
3. **Haptic Feedback (@capacitor/haptics)** – Provides vibration feedback to notify users of certain application events and improve the mobile experience.
4. **Google OAuth Login (@capgo/capacitor-social-login)** – Allows users to log in using their Google accounts. It is integrated into landing-page.component.ts, login.page.ts, and auth-modal.component.ts.

5. **Android Cleartext HTTP Configuration** – Configured in capacitor.config.ts using cleartext: true and androidScheme: 'http'. This allows the Android application to connect to the local XAMPP PHP/MySQL server through Wi-Fi during development and testing.

B. Data Handling

The application uses different methods to store, manage, and synchronize information between passengers, drivers, and the system.

1. **Reactive State Management and Data Persistence** – Uses RxJS BehaviorSubject in state.service.ts to manage bicycle information (bicycles$), bookings (bookings$), queues (queues$), and trips (trips$). Local storage is also used to retain application data.
2. **BroadcastChannel and Storage Events** – Uses BroadcastChannel with the channel name neust_bikego_sync_channel to synchronize booking and fleet updates across multiple tabs in the same browser. The application also uses browser storage events to detect changes in neust_bikego_app_state_v7 and neust_bikego_chat_messages.
3. **Cross-Device Synchronization** – Since BroadcastChannel and localStorage only work within the same browser environment, the application uses 2-second HTTP polling through the XAMPP MySQL backend to synchronize information between different devices. Functions include getBookingStatusXampp(), pollDriverRequestsXampp(), and getLiveTrackingXampp().
4. **Session Management and Authentication** – Uses sessionStorage and localStorage in auth.service.ts to manage user login sessions. The application prioritizes session data before checking local storage when restoring an active session. Login information is saved in both storage locations, allowing users to remain signed in after closing and reopening the application. Both storage locations are also cleared during logout.
5. **Atomic Booking Lock and Capacity Tracking** – Prevents multiple drivers from accepting the same booking through a server-side conditional database update. The application also tracks the number of booked passengers, with a maximum capacity of four seats per pedicab. Seat availability is automatically updated when a booking is accepted and restored when a trip is completed or cancelled.

C. API Operations

NEUST Bike Go uses a REST API connected to a PHP and MySQL backend through Angular's HttpClient, implemented in api.service.ts.

The application performs the following HTTP operations:

- **GET** – Retrieves user information, booking details, booking status, and other stored records.
- **POST** – Sends login information, creates new bookings, submits driver responses, and updates trip information.

The following PHP endpoints are used:

- register.php – Handles user registration.
- login.php – Handles user authentication.
- get_users.php – Retrieves user records.
- create_booking.php – Creates new ride bookings.
- get_booking_status.php – Retrieves booking status.
- driver_poll_requests.php – Checks incoming booking requests for drivers.
- driver_respond_booking.php – Processes driver responses to booking requests.
- update_trip_status.php – Updates the status of a trip.

The application also uses getLiveTrackingXampp() to retrieve live tracking information through the backend.

For map services, the application uses Leaflet version 1.9.4 through leaflet-map.service.ts, with Google Maps road tiles and OpenStreetMap as an alternative map source.

D. Important Implementation Choices

1. **Custom Campus Routing Using Dijkstra's Algorithm** – Uses a custom road network based on the NEUST Sumacab campus through campus-routing.service.ts. The system uses the Dijkstra algorithm and road snapping through snapToNearestRoadSegment() to calculate routes that follow campus roads instead of relying entirely on public road directions.
2. **Geofencing and Boundary Validation** – Uses PickupLocationService to restrict pickup pins to the NEUST Sumacab campus area. It also identifies designated pedicab terminals, particularly the Main Gate Stand and Architecture Stand.
3. **Atomic Booking and Seat Management** – Uses conditional SQL updates in driver_respond_booking.php to ensure that only one driver can accept a particular booking. It also automatically updates passenger capacity and prevents the number of passengers from exceeding the maximum of four seats.

4. **Real-Account-Only Fleet Management** – Uses registered user accounts and actual bicycle records instead of relying on fake or pre-seeded pedicab data in campus-data.ts.

5. **Bilingual Support (English and Filipino)** – Provides English and Filipino language options through language.service.ts, allowing members of the NEUST community to use the application in their preferred language.

6. Testing Results

Feature / Task Tested	Expected Result	Actual Result	Status
User Login and Registration	Authenticates users and directs them to the appropriate dashboard based on their account type.	Users can successfully register and log in to their respective accounts.	Pass
Ride Booking	Allows passengers to select pickup and drop-off locations and submit a ride request.	Booking requests are successfully created and booking information is displayed.	Pass
Pickup Location Pinning	Allows passengers to pin their preferred pickup location anywhere within the campus.	Users can select and pin locations on the campus map	Pass
Drop-off Location Selection	Allows passengers to select their desired destination within the campus.	Passengers can successfully select their preferred drop-off location.	Pass
Bike Origin Selection	Allows passengers to choose between the Main Gate and Arki as the bike's starting point.	Passengers can select their preferred bike origin.	Pass
Available Bike Viewing	Displays available bikes and queue information at the Main Gate and Arki.	Available bike information is displayed in the application.	Pass
Live Location Tracking	Displays the current location of the booked pedicab on the passenger's map.	Live tracking did not work properly during the first two tests but worked successfully during the third test.	Pass
Messaging and Calling Mobile Number	Allows passengers to communicate with their assigned drivers.	Passengers can use the messaging and calling features to contact drivers.	Pass
Notifications	Sends alerts about booking updates and driver arrival.	Notifications are successfully displayed for supported ride-related events.	Pass

Ride History	Displays previous rides, including bike number, driver name, fare, pickup location, and drop-off location.	Previous ride records and their corresponding details can be viewed.	Pass
Driver Booking Requests	Allows drivers to receive and accept passenger booking requests.	Drivers can view and accept available booking requests.	Pass
Passenger and Trip Details	Displays the passenger's name, pickup location, and drop-off location.	Passenger and trip information is displayed after accepting a booking.	Pass
Driver Messaging and Calling	Allows drivers to contact passengers for ride coordination.	Drivers can communicate with passengers through messaging and calling.	Pass
Driver Earnings Monitoring	Displays the driver's earnings from completed trips.	Drivers can view their recorded earnings.	Pass
Driver GPS and Map	Displays the driver's current location and provides map navigation.	The driver map and GPS functions can display location information.	Pass
Driver Online/Offline Status	Allows drivers to update their availability for incoming bookings.	Drivers can switch between online and offline status.	Pass
Admin Dashboard	Displays an overview of bikes, drivers, users, and trip information.	Dashboard information is displayed successfully.	Pass
Campus GPS and Location Management	Allows administrators to manage campus locations and designated pickup points.	Administrators can view and manage campus location information.	Pass
Bicycle Fleet Management	Displays registered bikes, assigned drivers, and availability information.	Bicycle and driver records can be viewed and managed.	Pass
Trip Monitoring	Allows administrators to monitor ongoing and completed trips.	Trip information and status can be viewed.	Pass
Passenger Demand Monitoring	Displays passenger demand at different campus locations.	Passenger demand information is displayed.	Pass
Demand Heatmap and Analytics	Shows areas with higher passenger demand through map-based information.	Demand information can be viewed through the analytics interface.	Pass
Maintenance Management	Allows administrators to view and manage reported bike maintenance issues.	Maintenance records and their status can be viewed.	Pass

Fare Management	Allows administrators to manage the standard transportation fare.	Fare information can be viewed and updated.	Pass
Broadcast Notices	Allows administrators to send announcements to passengers and drivers.	Notices can be created and displayed.	Pass
Cross-Device Booking Synchronization	Synchronizes booking information between passenger and driver devices.	During the initial test, booking information did not synchronize because localStorage stores data separately on each device. This was addressed in the succeeding tests.	

Observed Issues and Limitations

- **Map Coverage and Location Pinning**: Users initially experienced difficulty pinning locations in areas deeper inside the campus, particularly near CICT and CMBT, due to limitations in Google Maps coverage within the school.
- **Local Server Dependency:** When running the Android APK on a separate phone, the XAMPP backend requires both the phone and PC to be on the same Wi-Fi network with the correct PC IPv4 address.
- **Indoor GPS Accuracy:** GPS signal accuracy degrades inside buildings, which may slightly offset the initial pickup pin.

Recommended Improvements

- Improve the campus map by adding more detailed building locations and internal roads. Use a custom campus map and road network to make location pinning more accurate within the university.
- Cloud-Hosted Backend: Deploy the PHP/MySQL API to a live cloud host so students and drivers can sync bookings over mobile data without relying on a local Wi-Fi IP.
- Real-Time Updates: Add WebSocket or real-time listener support for instant driver-passenger status synchronization.

AI Usage Disclosure

The group used AI-powered tools, specifically ChatGPT and Anti Gravity, as assistants during the planning and development of NEUST Bike Go. These tools helped improve the application, identify additional features, solve technical problems, and fix errors encountered during development.

AI Tools Used

1. **ChatGPT (GPT)** – Used to suggest additional features and improvements that could be included in the application. It helped the group identify possible functionalities that would make NEUST Bike Go more useful and convenient for its intended users.

2. **Anti Gravity** – Used as a development assistant to help build and improve the application. It assisted in writing and modifying code, identifying and fixing bugs, and resolving technical issues. It also helped improve the application's geolocation functionality, particularly in making location pinning and GPS-related features work properly.

The group reviewed the suggestions and code provided by the AI tools to ensure that they matched the project's requirements. The generated code and proposed features were checked, tested, and modified when necessary. The group also conducted testing to verify the functionality of the application and adjusted based on the results.

AI tools served as development assistants, while the group remained responsible for the project's design, implementation decisions, testing, and final output.